UT1 · DESPLIEGUE DE APLICACIONES WEB — CFGS DAW

Control de versiones: repositorio local, ramas, fusiones y etiquetas

Actividad — PHP Words, una aplicación versionada con Git

Docente: José Manuel Martínez · IES Ramón Arcas Meca

OBJETIVO

Practicar la gestión de la historia de un proyecto con Git en un repositorio local. Desarrollarás una pequeña aplicación web que muestra las palabras reservadas de PHP y la ampliarás paso a paso (buscador, filtro por categorías, más palabras y documentación). **Lo evaluable no es la aplicación, sino cómo gestionas su historia**: qué guardas, cuándo, en qué rama y cómo unes el trabajo de varias ramas.

El código lo generará una IA a partir de los prompts que tú escribas. Todos los prompts deben respetar el contrato técnico que se describe más abajo, de modo que, aunque cada persona obtenga un código distinto, todas podréis completar las misiones.

> **Importante:** Este guion describe las **operaciones** que debes realizar sobre el repositorio, pero no te dice qué comandos escribir. Averiguarlos con tus apuntes, la documentación de Git y su ayuda integrada forma parte de la actividad. En la memoria deberás recoger los comandos que has usado.

REGLAS DE TRABAJO

- **Repositorio local:** sin servidor remoto, sin clonado y sin pull requests.
- **Rama principal:** se llama **main**. **Solo el primer commit se hace directamente en main.** Todo lo demás se hace en una rama propia y se incorpora a main fusionándola.
- **Entorno:** terminal Git Bash o Linux y VS Code (o similar). La aplicación se abre con doble clic en **index.html**.
- **Commits:** mensajes en español, en una frase que diga qué cambia. Cuando el guion indique un mensaje entre comillas, úsalo tal cual.
- **Hábito:** antes de guardar nada en el repositorio, consulta siempre el estado de los archivos.

ENTREGA: LA MEMORIA

Entregarás **una memoria en PDF** con el nombre **Apellidos_Nombre_UT1_PHPWords.pdf**: portada con tus datos, índice, una sección por misión y una reflexión final de 5 líneas. Cada misión debe recoger:

- **Comandos:** todos los que has escrito, en el orden real, copiados como texto (no como imagen) y en letra monoespaciada.
- **Prompts:** los que has enviado a la IA, completos y literales.
- **Capturas:** de **pantalla completa** (monitor entero, sin recortes), con la terminal o el navegador, la barra de tareas con fecha y hora y la ruta de la carpeta del proyecto visibles. Cada captura lleva un pie con su código (por ejemplo "Captura 3.2") y una frase que diga qué demuestra.
- **Respuestas:** a las preguntas "Observa" y "Concepto", con tus palabras y en dos o tres líneas.

Importante: Un comando sin captura que lo demuestre no puntúa. Una captura recortada, sin fecha o sin ruta visible tampoco. Cada misión indica qué capturas debes hacer.

CONTRATO TÉCNICO (OBLIGATORIO EN TODOS TUS PROMPTS)

Copia estas especificaciones, tal cual, en cada prompt. Si la IA propone otros nombres o estructuras, pídele que se ajuste.

Sistema de ficheros del proyecto

```
php-words/
├── index.html
├── README.md
├── notas.txt   (la creas a mano en la Misión 1)
├── css/
│   └── estilos.css
└── js/
     ├── config.js
     ├── palabras.js
     └── app.js
```

Reglas fijas

- **index.html:** carga los scripts en este orden exacto, **uno por línea**, al final del body: js/config.js, js/palabras.js y js/app.js. Sin módulos, sin librerías externas y sin fetch. Elementos con estos id: titulo (h1), subtitulo (p), version (span), controles (div vacío), contador (p) y lista (ul).
- **js/config.js:** exactamente tres constantes, una por línea: APP_TITULO = "PHP Words", APP_SUBTITULO = "Palabras reservadas de PHP" y APP_VERSION = "0.1.0".
- **js/palabras.js:** una única constante global PALABRAS, un array de objetos con palabra, categoria y descripcion. **Cada objeto ocupa una sola línea.** Categorías: control, funciones, clases, excepciones y otros. Mínimo 25 palabras.
- **js/app.js:** define la variable global **let filtros = { texto: "", categoria: "todas" };** y la función **aplicarFiltros()**, que muestra en #lista las palabras que cumplen los filtros y escribe en #contador "Mostrando N de M palabras". Al cargar, rellena título, subtítulo y versión desde config.js y llama a aplicarFiltros().
- **Funciones nuevas:** se entregan siempre como **un archivo nuevo** en js/ y **nunca modifican archivos existentes**. Solo usan filtros, aplicarFiltros(), PALABRAS y el elemento #controles. La línea script que las engancha la añades tú a mano.
- **Ejecución:** todo funciona abriendo index.html con doble clic.

Qué decides tú

Todo lo que no está en el contrato queda abierto: colores, tipografías, diseño (lista simple o tarjetas), texto de las descripciones, idioma de los comentarios, número de palabras y detalles de presentación en estilos.css. Aprovecha esa libertad para escribir un buen prompt.

- Se puntúa lo que está **demostrado en la memoria** con comandos, capturas completas y respuestas.

- Cada commit directo en main distinto del primero resta 0,5 puntos a la nota final.
- Marcas de conflicto olvidadas en el código final restan 0,5 puntos a la Misión 6.
- La nota final no baja de 0 ni supera 10.

Misión 1. El punto de partida · 1,0 puntos

- **Crea** una carpeta **php-words** y **sitúa** la terminal dentro de ella.
- **Escribe un prompt** para tu IA que cumpla el contrato y genere los 6 archivos. Debe incluir qué es la aplicación, el contrato completo y **al menos tres decisiones abiertas tuyas** (por ejemplo, paleta de colores, tono de las descripciones y número de palabras).
- **Guarda** los archivos con sus nombres exactos y **abre** index.html en el navegador.
- **Comprueba:** los id están bien escritos, cada objeto de PALABRAS ocupa una línea, config.js tiene tres líneas e index.html tiene exactamente tres líneas script. Si algo falla, pide a la IA que corrija solo eso.
- **Crea a mano** notas.txt con una línea: "Ideas: buscador, filtro por categoría, más palabras." Es tu bloc de notas y no formará parte de la aplicación.
- **Edita** README.md: título, una frase sobre la aplicación y una sección "Cómo ejecutar" con el texto "Pendiente de documentar."

Resultado esperado: la aplicación muestra el título, "v0.1.0", el contador con todas las palabras y la lista completa.

Capturas: *1.1 prompt enviado a la IA. 1.2 navegador con la aplicación. 1.3 editor con el árbol de archivos.*

Criterios: prompt que respeta el contrato y deja decisiones abiertas (0,5); aplicación funcionando y estructura correcta (0,5).

Misión 2. Mirar antes de guardar · 1,0 puntos

- **Inicializa** un repositorio local en la carpeta, con **main** como rama inicial, y **consulta el estado**.
- **Prepara** (zona de preparación) únicamente index.html, README.md y las carpetas css y js, con una sola orden. **notas.txt debe quedarse fuera.**
- **Consulta el estado** en formato abreviado (una línea por archivo).
- **Confirma** el primer commit con el mensaje "Estructura inicial de la aplicación PHP Words". Es el único commit que se hace directamente en main.
- **Consulta el historial** en formato resumido (un commit por línea).

Observa: ¿qué archivos aparecen "sin seguimiento" y qué significa? Tras preparar los archivos, ¿qué letra y color tiene cada uno y en qué estado sigue notas.txt?

Concepto: dibuja los tres espacios de Git y coloca en cada uno los archivos de tu proyecto en este momento, incluido notas.txt. Incluye el dibujo en la memoria.

```
TRABAJO  --preparar-->  PREPARACIÓN  --confirmar-->  REPOSITORIO
(archivos que editas)   (staging)                    (.git, historial)

Comparaciones posibles entre zonas:
  1. trabajo  <->  preparación
  2. preparación  <->  último commit
  3. trabajo  <->  último commit
```

Capturas: *2.1 estado justo después de crear el repositorio. 2.2 estado abreviado tras preparar. 2.3 historial resumido tras el commit.*

Criterios: repositorio creado, estado consultado y preparación selectiva sin notas.txt (0,4); primer commit e historial (0,3); respuestas "Observa" y dibujo de las tres zonas (0,3).

Misión 3. Documentación en su propia rama · 2,0 puntos

- **Crea** una rama **documentacion** y **cámbiate** a ella en un solo paso.
- **Edita** README.md: sustituye "Pendiente de documentar." por una instrucción real, por ejemplo "Abrir index.html en el navegador (doble clic)."
- **Muestra las diferencias** entre el directorio de trabajo y la zona de preparación.
- **Prepara** README.md. Sin confirmar, **edita de nuevo** README.md y añade al final una segunda línea, por ejemplo "Requiere un navegador moderno."
- **Consulta el estado** en formato abreviado.
- **Compara** tres veces, una captura por comparación: (a) directorio de trabajo con zona de preparación; (b) zona de preparación con último commit; (c) directorio de trabajo con último commit.
- **Prepara** de nuevo README.md y **confirma** con un mensaje deliberadamente descuidado: "Documenta ejecucion".
- **Modifica el mensaje del último commit** sin crear uno nuevo, para que diga "Documenta cómo ejecutar la aplicación".
- **Muestra el contenido** del último commit con un resumen de los archivos que cambia.

Observa: en la primera comparación, ¿qué líneas llevan - y cuáles +? Junto a README.md aparece "MM": explica con tus palabras por qué hay dos letras M.

Concepto: explica qué ves en cada comparación y por qué la tercera es la suma de las otras dos. ¿Por qué es seguro reescribir el último commit aquí y qué problema habría si ya se hubiera compartido?

Capturas: *3.1 diferencias iniciales. 3.2 estado con MM. 3.3, 3.4 y 3.5 una por comparación. 3.6 confirmación, corrección del mensaje e historial resumido. 3.7 resumen del último commit.*

Criterios: rama y diferencias iniciales interpretadas (0,5); MM explicado y tres comparaciones justificadas (0,7); commit y corrección del mensaje (0,5); resumen del commit y respuesta sobre reescribir la historia (0,3).

Misión 4. Volver a main e incorporar la documentación · 0,5 puntos

- **Cámbiate** a main y **abre** README.md: todavía no contiene tus cambios.
- **Fusiona** la rama documentacion en la rama actual.
- **Muestra el historial** de todas las ramas como grafo, una línea por commit y con las referencias visibles.

Observa: ¿dónde estaban los cambios del README antes de fusionar? Git indica "Fast-forward": ¿qué significa y por qué no se crea un commit nuevo?

Capturas: *4.1 README abierto en main sin los cambios. 4.2 salida de la fusión. 4.3 grafo del historial.*

Criterios: fusión realizada desde main y demostrada (0,25); explicación del fast-forward y del grafo (0,25).

Misión 5. Tres ideas en paralelo · 2,0 puntos

> **Importante:** Las tres ramas deben salir de main **antes** de fusionar ninguna. Termina cada rama volviendo a main. Si creas una rama estando en otra, la Misión 6 no tendrá conflicto y perderás el objetivo de la actividad.

5.A · Rama mas-palabras (0,5)

- **Crea** la rama **mas-palabras** desde main y **cámbiate** a ella.
- **Pide** a la IA tres palabras reservadas nuevas (por ejemplo abstract, static y const), con el mismo formato de una línea por objeto, y **añádelas** al final del array de js/palabras.js. Comprueba en el navegador que el contador sube en tres.
- **Consulta el estado**, **prepara** solo js/palabras.js y **confirma** con el mensaje "Añade más palabras reservadas". **Vuelve** a main.

5.B · Rama buscador (0,75)

- **Crea** la rama **buscador** desde main y **cámbiate** a ella.
- **Escribe un prompt** nuevo para generar **un único archivo**, js/buscador.js: crea un campo de texto dentro de #controles y, al escribir, actualiza filtros.texto y llama a aplicarFiltros(). Incluye el contrato; deja abiertos el placeholder y los estilos.
- **Edita a mano** index.html: añade la línea script de js/buscador.js **justo debajo** de la de app.js.
- **Edita a mano** js/config.js: cambia la versión a "0.2.0".
- **Prueba** la aplicación (escribe "for" y comprueba que el contador cambia) y **consulta el estado**.
- **Prepara** index.html y la carpeta js por nombre, **sin usar la orden que lo prepara todo**, y **confirma** con el mensaje "Añade buscador por texto". **Vuelve** a main.

```
php-words/   (estado esperado en la rama buscador antes de confirmar)
├── index.html      [MODIFICADO: nueva línea script]
├── README.md
├── notas.txt       [SIN SEGUIMIENTO: no se añade]
├── css/
│   └── estilos.css
└── js/
     ├── config.js    [MODIFICADO: versión 0.2.0]
     ├── palabras.js
     ├── app.js
     └── buscador.js  [NUEVO]
```

Observa: en el estado aparecen archivos modificados y archivos sin seguimiento. ¿Cuáles son unos y cuáles otros? ¿Por qué prepararlos por nombre en lugar de preparar todo de golpe?

5.C · Rama categorias (0,75)

- **Crea** la rama **categorias** desde main y **cámbiate** a ella.
- **Escribe otro prompt** para generar **un único archivo**, js/categorias.js: crea un desplegable dentro de #controles con la opción "todas" y las categorías existentes en PALABRAS y, al cambiar, actualiza filtros.categoria y llama a aplicarFiltros().
- **Edita a mano** index.html: añade la línea script de js/categorias.js **justo debajo** de la de app.js. **Edita a mano** js/config.js: versión "0.3.0".

- **Prueba** el desplegable, **prepara** index.html y js y **confirma** con el mensaje "Añade filtro por categoría".
- **Vuelve** a main y **muestra el historial** de todas las ramas como grafo.

```
* 8d1f0aa (categorias) Añade filtro por categoría
| * 3c77b12 (buscador) Añade buscador por texto
|/
| * 5a09e41 (mas-palabras) Añade más palabras reservadas
|/
* 21b2a0e (HEAD -> main, documentacion) Documenta cómo ejecutar la aplicación
* 7e46a2c Estructura inicial de la aplicación PHP Words
```

Resultado esperado: un grafo como el anterior (los identificadores serán distintos), con las tres ramas saliendo del mismo commit.

Capturas: *5.1 prompt del buscador. 5.2 estado con modificados y sin seguimiento. 5.3 aplicación con el buscador. 5.4 prompt de categorías. 5.5 aplicación con el desplegable. 5.6 grafo con las tres ramas. Incluye el árbol de archivos de cada rama.*

Criterios: mas-palabras: rama, cambio, preparación selectiva y commit (0,5); buscador: prompt con contrato, archivo nuevo, línea script, versión 0.2.0 y commit (0,75); categorias: los mismos elementos con versión 0.3.0, y las tres ramas parten del mismo commit (0,75).

Misión 6. Fusionar, etiquetar y resolver · 2,5 puntos

- Estando en main, **fusiona** la rama buscador. **Crea una etiqueta ligera** v0.2.0 en el commit actual y **lista** las etiquetas.
- **Fusiona** la rama mas-palabras indicando tú el mensaje del commit de fusión: "Fusiona mas-palabras". **Muestra el historial** como grafo, con referencias.
- **Fusiona** la rama categorias. **Git se detendrá con conflictos: es el objetivo de la misión.**
- **Consulta el estado** para localizar los archivos en conflicto y **muestra los commits** de ambas ramas implicados en el conflicto (el historial tiene una opción pensada para este momento).
- **Abre** cada archivo en conflicto, **decide** qué versión conservar, **borra** todas las marcas y deja el archivo limpio y funcional.
- **Comprueba** en el navegador que funcionan a la vez el buscador y el desplegable y que la versión mostrada es correcta.
- **Marca como resueltos** los archivos corregidos, **confirma** la fusión con el mensaje "Fusiona categorias y resuelve conflictos" y **muestra el historial** como grafo.

Marcas de conflicto: así se leen en el archivo.

```
<<<<<<< HEAD         <- inicio: versión de main (la tuya)
línea o líneas de main
=======                <- separador
línea o líneas de la rama que fusionas
>>>>>>> categorias   <- fin del conflicto
```

Observa: ¿es Fast-forward la fusión de buscador y por qué? La de mas-palabras crea un commit de fusión

sin conflicto: ¿por qué no hay conflicto y por qué sí se crea un commit?

Decide: en **index.html** la aplicación necesita ambas funciones: ¿qué haces con las dos líneas script? En **js/config.js** solo puede haber una versión: ¿cuál es la correcta para el estado actual del proyecto y por qué? Ayúdate de las etiquetas y del historial.

Concepto: ¿por qué hubo conflicto en index.html y config.js pero no en palabras.js? Respóndelo con la idea de "dos ramas que modifican las mismas líneas desde un mismo punto de partida".

Capturas: *6.1 fusión de buscador y etiqueta. 6.2 fusión de mas-palabras y grafo. 6.3 fusión de categorias con el mensaje CONFLICT. 6.4 estado y commits implicados. 6.5 index.html con marcas. 6.6 config.js con marcas. 6.7 ambos archivos resueltos. 6.8 aplicación funcionando con la versión correcta. 6.9 commit final y grafo.*

Criterios: fusión de buscador, etiqueta v0.2.0 y explicación (0,5); fusión de mas-palabras a tres bandas y explicación (0,5); conflicto provocado y localizado (0,5); index.html resuelto con ambas líneas y sin marcas (0,4); config.js con la versión correcta y razonamiento (0,4); commit final, comprobación funcional y concepto (0,2).

Misión 7. Etiquetas y limpieza · 1,0 puntos

- **Crea una etiqueta anotada** v0.3.0 en el estado actual, con el mensaje "Versión 0.3.0: buscador, categorías y más palabras".
- **Consulta el historial** resumido, localiza el identificador del primer commit y **crea una etiqueta ligera** v0.1.0 que apunte a él en lugar del commit actual.
- **Lista** las etiquetas junto con la primera línea de su mensaje y **muestra el contenido** de la etiqueta v0.3.0.
- **Elimina** las cuatro ramas ya fusionadas (documentacion, mas-palabras, buscador y categorias) con el borrado seguro, que solo permite eliminar ramas ya fusionadas, y **lista** las ramas que quedan.
- **Consulta el estado** y **muestra el historial** final de todas las ramas como grafo, con referencias.

```
php-words/   (estado final en main)
├── .git/          el repositorio (no se toca a mano)
├── index.html     tres scripts + buscador + categorias
├── README.md      con "Cómo ejecutar"
├── notas.txt      sin seguimiento (nunca se añadió)
├── css/
│   └── estilos.css
└── js/
     ├── config.js    versión "0.3.0"
     ├── palabras.js  con las tres palabras nuevas
     ├── app.js
     ├── buscador.js
     └── categorias.js
```

Observa: ¿qué información tiene la etiqueta anotada que no tiene la ligera? ¿Sigue notas.txt sin seguimiento? Explica por qué es correcto que nunca se guardara.

Capturas: *7.1 creación de las etiquetas v0.3.0 y v0.1.0. 7.2 listado de etiquetas con su mensaje. 7.3*

contenido de v0.3.0. 7.4 borrado y listado de ramas. 7.5 estado con notas.txt. 7.6 grafo final. 7.7 aplicación final funcionando.

Criterios: etiqueta anotada y respuesta sobre sus diferencias (0,25); etiqueta retroactiva v0.1.0 (0,25); ramas fusionadas eliminadas con el borrado seguro (0,25); estado final con notas.txt explicada y grafo final (0,25).

REFLEXIÓN FINAL Y AYUDA

Cierra la memoria con una reflexión de cinco líneas: qué parte del trabajo con Git te ha resultado más difícil y qué harías distinto en un proyecto real. No suma puntos, pero una memoria sin ella se considera incompleta.

- **Si te quedas bloqueado:** consulta el estado del repositorio; Git casi siempre indica el siguiente paso.
- Si la IA ha tocado un archivo que no debía, **descarta los cambios no guardados** de ese archivo y vuelve a pedírselo respetando el contrato.
- Si una fusión se complica, puedes **abortarla** y volver al estado anterior para repetirla.
- Usa la ayuda integrada de Git (git help seguido del nombre de la operación) y tus apuntes.

Verificación: Entrega la memoria en PDF por la vía indicada por el docente. Si ves "detached HEAD" o un error que no entiendes, no sigas: avisa al docente.

La memoria se debe entregar con el siguiente nombre de fichero "*Apellidos_Nombre_UTX_PHPWords.pdf*"